SereneDB challenge · SchubKI Hackathon 2026

Making Sense of Logs

Ask your logs a question. Get the answer your role needs, with proof.

Amirali & Konark

5 M
log lines

3
roles

2
commands to set up

THE PROBLEM

Logs hold the answers. Nobody reads them.



Engineer

“What failed?”



Team lead

“What keeps coming back?”



Manager

“What do we fix first?”

Millions of lines don't fit in a prompt, and an AI that guesses numbers can't be trusted.

THE PROBLEM

Raw logs mislead

855,501

FATAL lines, but only 36,714 bursts of problems

1 / 10

real alerts in the top 10 when ranked by log level

69.6 %

of claims correct when an AI greps the raw files (pilot)

THE SOLUTION

The database counts, the AI explains

Ask

in plain words

SereneDB

searches and counts every line

Jev

knows which messages are real faults

Answer

for your role, with proof

```
docker compose up -d  ·  claude  ·  “Load the base corpus”
```

DEMO

One problem, three answers

Engineer

1,024 nodes hit by a new fault

Next: check host 172.16.96.116

Team lead

31 bursts, owner disputed

Next: two teams triage together

Manager

Rank 1 of 133 problems

Next: name one owner now

WHY IT'S BETTER

More accurate and faster

62 / 62

numbers traced to a query

13 – 2

wins vs the reference design (blind judge)

13 s

per answer, vs 71 s for an AI grepping raw files (pilot)

HOW IT FITS THE COMPANY

It plugs into routines that already exist

On-call

the engineer, during an incident

Weekly review

the lead, recurring problems by team

Monthly planning

the manager, a computed ranking

Their tools
Claude Code / any MCP client

Their data
stays on their machine

Their logs
any file format

Their teams
access per team

CHEAPER THAN THE ALTERNATIVES

Judge once, not every line

An LLM reads every line

≈ 177 M tokens

every time (estimate)

Jev judges 1,126 patterns

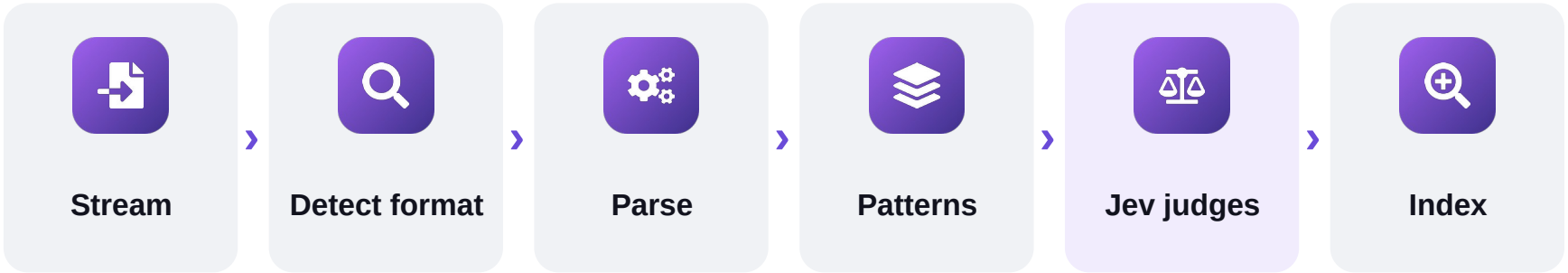
$0.02 · 20 s

once, reused by every answer

real alerts in the top 10: 1 → 5

LOADING

One sentence loads your logs



72 s

base corpus, 5 M lines

1.6 s

a new 21 k-line file

76

log formats detected

PERFORMANCE

Built on SereneDB's search-analytics engine

Inverted index

7 ms

full-text count, 4.7 M lines

Term dictionary

3–39 ms

group-bys without scanning

SQL-native load

18 s

4.7 M lines parsed

ASOF · ROLLUP · PIVOT

< 100 ms

timelines and rankings

Remote Parquet

2.8 s

1 B rows on S3, in place

On-demand index

0.94 ms

vs 281 ms scan

One MCP server, four ways to use it

44 tools

just ask

5 prompts

type /

6 resources

type @

Completions

press Tab

44 tools, grouped by the question

Find

`search_logs` · `get_context`

Count

`count_by` · `trend` · `rollup`

Recurring

`top_patterns` · `pattern_history`

Explain

`burst_timeline` · `precursors`

Decide

`priority_table` · `owner_of`

Set up

`load_logs` · `use_dataset`

Every result: its SQL, the exact total, and a flag if cut to 50 rows

Shortcuts in Claude Code

Prompts /

- `investigate_failure`
- `review_recurring_problems`
- `set_priorities`
- `worked_investigation`
- `whole_corpus_counts`

Resources @

- `logs://datasets`
- `logs://glossary`
- `logs://field-guide/{dataset}`
- `logs://ownership/{dataset}`
- `logs://session/latest`

WOW

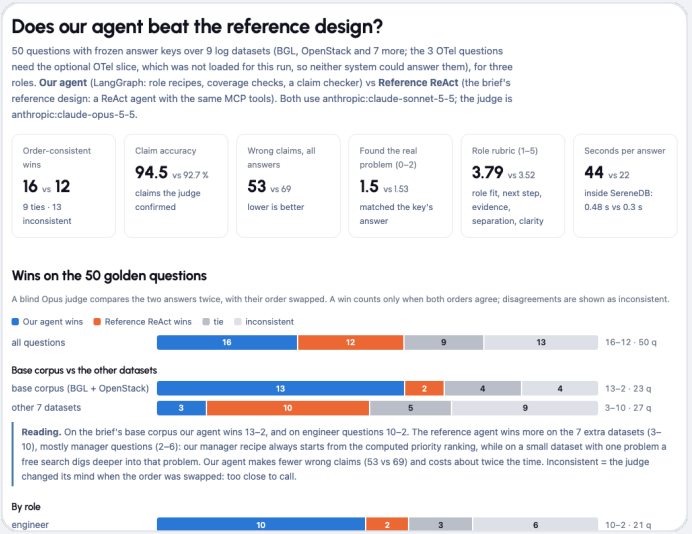
Beyond what the brief asked for

- Your own logs
- Verify button
- 1 B remote rows
- Access per team
- Blind evaluation
- Judged once

Every answer comes with its proof



Session report



Eval dashboard

SCALE

Where it goes next

Mittelstand

production and IT logs

Other industries

anything that writes logs

A product

a self-hosted log copilot

10

log types, no code change

11.2 M

lines loaded in 47 s

1 B

remote rows, in place

Try it in two commands

```
docker compose up -d
claude
> Load the base corpus
```

github.com/aircode610/making-sense-of-logs-schubki-ai-hackathon